Algorytmy sortowania

Matura rozszerzona z informatyki – Sesja 5

Kacper Gębusia

Plan sesji

1. Wprowadzenie: inwersje, stabilność, sortowanie w miejscu
2. Sortowanie bąbelkowe
3. Sortowanie kubełkowe
4. Sortowanie przez wybieranie
5. Sortowanie przez wstawianie
6. Quicksort (wersja z listami i w miejscu)
7. Sortowanie przez scalanie
8. Porównanie algorytmów, sortowanie w Pythonie, biblioteki
9. Ćwiczenia

Powtórka z Sesji 4

1. Podaj złożoność czasową (notacja *O*) i krótko uzasadnij:

a)

```
for i in range(n):
    for j in range(n):
        x += 1
```

b)

```
i = 1
while i < n:
    i *= 2
```

2. Rozważ poniższą funkcję w Pythonie:

```
def g(n):
    if n == 0:
        return 0
    return n + g(n - 1)
```

a) Co zwróci wywołanie `g(4)`? Prześledź kolejne wywołania.
b) Ile razy zostanie wywołana funkcja `g` (łącznie z wywołaniem początkowym `g(4)`)?

Po co sortować? Podstawowe pojęcia

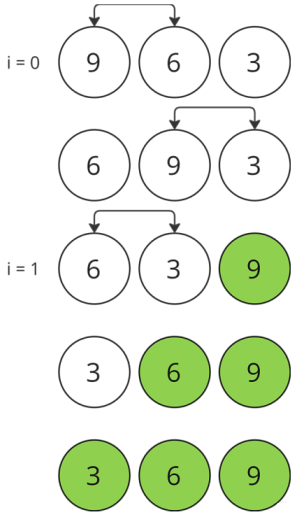
Sortowanie – ustawienie elementów w porządku (np. rosnąco). Posortowane dane pozwalają m.in. na wyszukiwanie binarne (Sesja 6), łatwe znajdowanie mediany, duplikatów, „pierwszych k" elementów.

- **Inwersja** – para pozycji $i < j$, dla której $a[i] > a[j]$. Lista posortowana rosnąco: 0 inwersji; malejąco: $\frac{n(n-1)}{2}$.
- **Sortowanie stabilne** – elementy o równych kluczach zachowują wzajemny porządek z wejścia.
- **Sortowanie w miejscu (in-place)** – algorytm używa tylko $O(1)$ (lub niewiele) dodatkowej pamięci.

Dane przykładowe: $A = [5, 2, 4, 6, 1, 3]$, inwersji: $4 + 1 + 2 + 2 = \mathbf{9}$.

Sortowanie bąbelkowe – idea

Wielokrotnie przechodzimy przez listę i zamieniamy **sąsiednie** elementy, jeśli są w złej kolejności. Po każdym przebiegu największy z nieposortowanych „wypływa” na koniec.



Sortowanie bąbelkowe – kod

```
def bubble_sort(a):
    n = len(a)
    for i in range(n - 1):
        zamiana = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                zamiana = True
        if not zamiana:       # brak zamian -> koniec
            break
    return a
```

Bąbelkowe – przebieg i złożoność

$A = [5, 2, 4, 6, 1, 3]$:

Przebieg	Stan listy	Zamian
start	5 2 4 6 1 3	–
1	2 4 5 1 3 **6**	4
2	2 4 1 3 **5 6**	2
3	2 1 3 **4 5 6**	2
4	1 2 3 4 5 6	1
5	1 2 3 4 5 6	0 $\rightarrow$ koniec

Razem 9 zamian = liczba inwersji. Porównań: $5 + 4 + 3 + 2 + 1 = 15$.

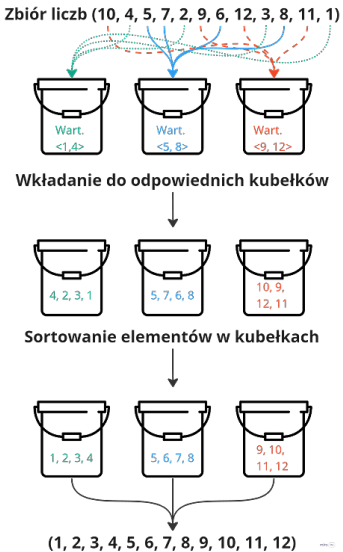
Złożoność:

- najgorszy (lista malejąca): $\frac{n(n-1)}{2}$ porównań i zamian $\rightarrow O(n^2)$
- najlepszy (lista posortowana, z flagą): $n - 1$ porównań $\rightarrow O(n)$
- pamięć $O(1)$, stabilne, w miejscu

Sortowanie kubełkowe – idea

Nie porównujemy elementów całej listy między sobą. Dzielimy zakres wartości na przedziały równej szerokości – **kubełki** – i wrzucamy każdy element do kubełka wyznaczonego przez wartość (`x // szerokosc`), sortujemy kubełki osobno (tu: bąbelkowo) i sklejamy je po kolei.

Założenie: liczby całkowite nieujemne, w miarę równomiernie rozłożone.



Sortowanie kubełkowe – kod

```
def bucket_sort(a, szerokosc=10):
    if not a:
        return a
    liczba = max(a) // szerokosc + 1        # ile kubelkow
    kubelki = [[] for _ in range(liczba)]
    for x in a:
        kubelki[x // szerokosc].append(x)   # do kubelka
    wynik = []
    for k in kubelki:
        wynik += bubble_sort(k)             # sortuj i doklej
    return wynik
```

Kubełkowe – przebieg i złożoność

$A = [29, 25, 3, 49, 9, 37, 21, 43]$, szerokość 10 ($\lfloor 49/10 \rfloor + 1 = 5$ kubełków):

Kub.	Przedział	Po wrzuceniu	Po sort.
0	0–9	3, 9	3, 9
1	10–19	(pusty)	(pusty)
2	20–29	29, 25, 21	21, 25, 29
3	30–39	37	37
4	40–49	49, 43	43, 49

Wynik: $[3, 9, 21, 25, 29, 37, 43, 49]$

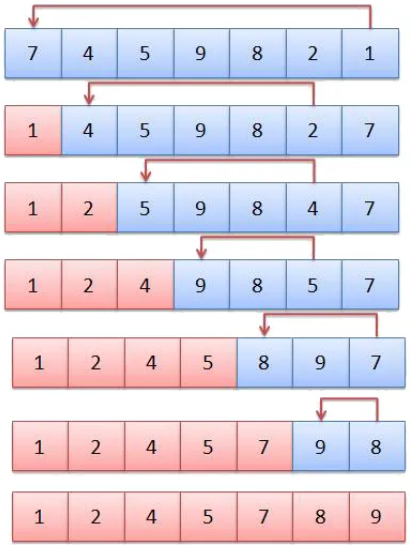
Złożoność (n elementów, k kubełków):

- rozkład równomierny: średnio $O(n + k)$ – liniowo
- najgorszy: wszystko w jednym kubełku, malejąco, np. $[36, 35, 34, 33, 32, 31]$ $\rightarrow O(n^2)$
- pamięć $O(n + k)$, stabilne

Nie opłaca się, gdy zakres jest ogromny względem n: dla $[3, 5, 1000000]$ powstaje 100001 kubełków, prawie wszystkie puste.

Sortowanie przez wybieranie – idea

W każdym kroku znajdujemy **najmniejszy** element nieposortowanej części i zamieniamy go z pierwszym elementem tej części.



Sortowanie przez wybieranie – kod

```
def selection_sort(a):
    n = len(a)
    for i in range(n - 1):
        m = i                       # indeks najmniejszego
        for j in range(i + 1, n):
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
    return a
```

Wybieranie – przebieg i złożoność

$A = [5, 2, 4, 6, 1, 3]$:

Krok i	Minimum (indeks)	Stan po zamianie
0	1 (idx 4)	1 2 4 6 5 3
1	2 (idx 1)	1 2 4 6 5 3
2	3 (idx 5)	1 2 3 6 5 4
3	4 (idx 5)	1 2 3 4 5 6
4	5 (idx 4)	1 2 3 4 5 6

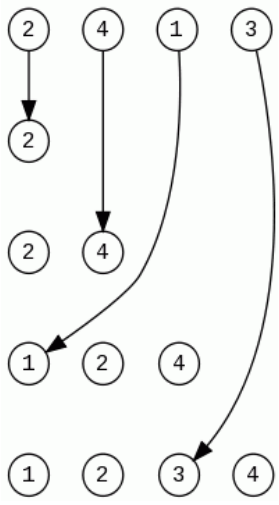
Złożoność:

- zawsze $\frac{n(n-1)}{2}$ porównań (tu 15) $\rightarrow O(n^2)$, nawet dla listy posortowanej
- zamian co najwyżej $n - 1$ (tu 3)
- pamięć $O(1)$

Niestabilne: dla $[2_a, 2_b, 1]$ minimum (1) zamienia się z $2_a \rightarrow [1, 2_b, 2_a]$.

Sortowanie przez wstawianie – idea

Jak układanie kart w ręce: lewa część jest posortowana; kolejny element („klucz”) wstawiamy na właściwe miejsce, przesuwając większe elementy w prawo.



Sortowanie przez wstawianie – kod

```
def insertion_sort(a):
    for i in range(1, len(a)):
        klucz = a[i]
        j = i - 1
        while j >= 0 and a[j] > klucz:
            a[j + 1] = a[j]          # przesuniecie w prawo
            j -= 1
        a[j + 1] = klucz
    return a
```

Wstawianie – przebieg i złożoność

$A = [5, 2, 4, 6, 1, 3]$:

i	Klucz	Stan po wstawieniu	Przesunięć
1	2	2 5 4 6 1 3	1
2	4	2 4 5 6 1 3	1
3	6	2 4 5 6 1 3	0
4	1	1 2 4 5 6 3	4
5	3	1 2 3 4 5 6	3

Razem 9 przesunięć = liczba inwersji.

Złożoność:

- najgorszy (malejąca) i średni: $O(n^2)$
- **najlepszy (posortowana):** $O(n)$
- pamięć $O(1)$, stabilne

Najlepszy z prostych algorytmów dla list **prawie posortowanych**.

Quicksort – wersja z listami

Wybieramy **pivot** i dzielimy listę na elementy mniejsze, równe i większe. Mniejsze i większe sortujemy rekurencyjnie i sklejamy.

```
def quicksort(a):
    if len(a) <= 1:
        return a
    pivot = a[len(a) // 2]                    # element srodkowy
    mniejsze = [x for x in a if x < pivot]
    rowne    = [x for x in a if x == pivot]
    wieksze  = [x for x in a if x > pivot]
    return quicksort(mniejsze) + rowne + quicksort(wieksze)
```

To wersja czytelna, ale tworzy nowe listy (nie jest „w miejscu”).

Quicksort – przebieg i złożoność

$A = [5, 2, 4, 6, 1, 3]$:

1. pivot $= a[3] = 6$: mniejsze $= [5, 2, 4, 1, 3]$, równe $= [6]$, większe $= [\,]$
2. `quicksort([5,2,4,1,3])`: pivot $= 4$: mniejsze $= [2, 1, 3]$, równe $= [4]$, większe $= [5]$
3. `quicksort([2,1,3])`: pivot $= 1$: mniejsze $= [\,]$, równe $= [1]$, większe $= [2, 3]$; `quicksort([2,3])` $\rightarrow [2, 3]$. Wynik: $[1, 2, 3]$
4. Sklejamy: $[1, 2, 3] + [4] + [5] = [1, 2, 3, 4, 5]$, a na końcu $+[6]$: $[1, 2, 3, 4, 5, 6]$

Złożoność:

- dobry pivot (dzieli mniej więcej na pół): $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$
- zły pivot (zawsze min lub max): $T(n) = T(n-1) + O(n) \rightarrow$ **najgorszy** $O(n^2)$
- średnio $O(n \log n)$; losowy pivot utrudnia trafienie w najgorszy przypadek

Quicksort w miejscu – podział (Lomuto)

Pracujemy na jednej tablicy, elementy **zamieniamy** w niej miejscami.

- pivot to **ostatni** element `a[hi]`; `i` – indeks ostatniego „małego” (<= pivot), na początku `lo - 1`
- dla `j` od `lo` do `hi-1`: jeśli `a[j] <= pivot`, zwiększ `i` i zamień `a[i]` z `a[j]`
- na końcu zamień pivot z `a[i+1]` – pivot trafia na swoje miejsce

```
def podzial(a, lo, hi):
    pivot = a[hi]
    i = lo - 1
    for j in range(lo, hi):
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[hi] = a[hi], a[i + 1]    # pivot na miejsce
    return i + 1
```

Quicksort w miejscu – kod i przebieg

```
def quicksort_w_miejscu(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo < hi:
        p = podzial(a, lo, hi)
        quicksort_w_miejscu(a, lo, p - 1)
        quicksort_w_miejscu(a, p + 1, hi)
    return a
```

Pierwszy podział $A = [5, 2, 4, 6, 1, 3]$ (pivot $= a[5] = 3$, $i = -1$):

j	$a[j] \leq 3$?	i	Stan tablicy
0	nie	-1	5 2 4 6 1 3
1	tak, zamiana $a[0] \leftrightarrow a[1]$	0	2 5 4 6 1 3
2, 3	nie	0	2 5 4 6 1 3
4	tak, zamiana $a[1] \leftrightarrow a[4]$	1	2 1 4 6 5 3
koniec	pivot: zamiana $a[2] \leftrightarrow a[5]$		2 1 **3** 6 5 4

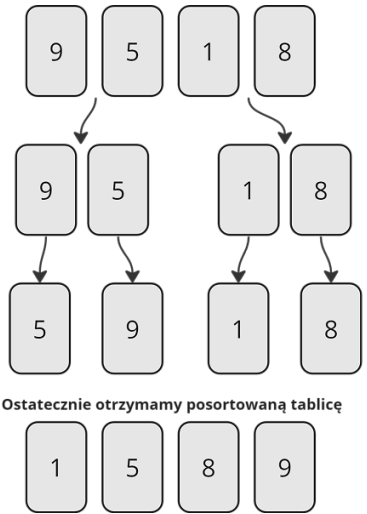
Zwraca indeks 2: po lewej $[2, 1]$, po prawej $[6, 5, 4]$.

Quicksort w miejscu – złożoność

- czas jak wcześniej: średnio $O(n \log n)$, najgorszy $O(n^2)$
- brak nowych list – dodatkowa pamięć to stos rekurencji: średnio $O(\log n)$, w najgorszym przypadku $O(n)$
- **niestabilny:** dla $[2_a, 2_b, 1]$ pierwszy podział (pivot $= 1$) daje $[1, 2_b, 2_a]$
- **uwaga:** dla listy już posortowanej pivot (ostatni element) jest zawsze największy $\rightarrow$ najgorszy przypadek $O(n^2)$. Dlatego w praktyce pivot losuje się lub wybiera środkowy element.

Sortowanie przez scalanie (mergesort) – idea

Dzielimy listę na dwie połowy, sortujemy rekurencyjnie, a potem **scalamy** dwie posortowane listy, porównując ich „czołowe” elementy.



Sortowanie przez scalanie (mergesort) – kod

```
def merge(l, p):
    wynik = []
    i = j = 0
    while i < len(l) and j < len(p):
        if l[i] <= p[j]:
            wynik.append(l[i])
            i += 1
        else:
            wynik.append(p[j])
            j += 1
    return wynik + l[i:] + p[j:]       # doklejenie reszty

def merge_sort(a):
    if len(a) <= 1:
        return a
    s = len(a) // 2
    return merge(merge_sort(a[:s]), merge_sort(a[s:]))
```

Scalanie – przebieg i złożoność

$A = [5, 2, 4, 6, 1, 3]$:

- podział: $[5, 2, 4]$ i $[6, 1, 3]$; dalej $[5]$, $[2, 4]$ oraz $[6]$, $[1, 3]$
- scalanie w górę: $[2], [4] \to [2, 4]$; $[5], [2, 4] \to [2, 4, 5]$; $[1], [3] \to [1, 3]$; $[6], [1, 3] \to [1, 3, 6]$
- końcowe scalenie $[2, 4, 5]$ i $[1, 3, 6]$: 2 vs 1 $\to$ 1; 2 vs 3 $\to$ 2; 4 vs 3 $\to$ 3; 4 vs 6 $\to$ 4; 5 vs 6 $\to$ 5; reszta $[6]$ (5 porównań)

Złożoność:

- $\log_2 n$ poziomów, scalanie na poziomie $O(n) \to$ **zawsze** $O(n \log n)$; $T(n) = 2T(n/2) + O(n)$
- pamięć dodatkowa $O(n)$
- stabilne (dzięki <= przy scalaniu)

Porównanie algorytmów sortowania

Algorytm	Najlepszy	Średni	Najgorszy	Pamięć	Stabilny
Bąbelkowe (z flagą)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	tak
Kubełkowe	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	tak
Przez wybieranie	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	nie
Przez wstawianie	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	tak
Szybkie (listy)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$	tak
Szybkie (w miejscu)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	nie
Przez scalanie	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	tak

Który wybrać? liczby całkowite z małego, znanego zakresu $\rightarrow$ kubełkowe; lista prawie posortowana $\rightarrow$ wstawianie; gwarancja $O(n \log n)$ $\rightarrow$ scalanie; szybko w praktyce i mało pamięci $\rightarrow$ quicksort w miejscu; mało pamięci i mało zamian $\rightarrow$ wybieranie.

Sortowanie wbudowane w Pythonie

```
a = [5, 2, 4, 6, 1, 3]
print(sorted(a))                  # [1, 2, 3, 4, 5, 6]  (nowa lista)
a.sort()                          # sortuje liste w miejscu
print(sorted(a, reverse=True))    # [6, 5, 4, 3, 2, 1]
print(sorted(["kot", "ala", "pies", "ma"], key=len))
                                  # ['ma', 'kot', 'ala', 'pies']
```

`sorted` i `.sort()` mają złożoność $O(n \log n)$ i są **stabilne** (słowa `kot` i `ala` zostają w oryginalnej kolejności). W zadaniach praktycznych zwykle używamy sortowania wbudowanego, o ile polecenie nie wymaga własnej implementacji – zawsze czytamy polecenie.

Ciekawostka: sortowania w bibliotekach

Żaden język nie używa „czystego” bąbelkowego ani wybierania. Biblioteki łączą algorytmy: szybkie lub scalające dla dużych danych, wstawianie dla małych fragmentów, czasem kopcowanie jako zabezpieczenie.

Język / funkcja	Algorytm	Stabilne
Python: `sorted`, `.sort()`	**Timsort** (scalanie + wstawianie)	tak
Java: `Arrays.sort(int[])`	dwupivotowy quicksort	–
Java: `Arrays.sort(Object[])`	Timsort	tak
C++: `std::sort`	zwykle **introsort** (quicksort + kopcowanie + wstawianie)	nie
C++: `std::stable_sort`	wariant sortowania przez scalanie	tak
JavaScript (V8): `sort`	Timsort	tak
Rust: `sort` / `sort_unstable`	driftsort / ipnsort	tak / nie
Go: `sort.Slice`, `slices.Sort`	pdqsort	nie

Wniosek: nie ma jednego najlepszego algorytmu – dobre biblioteki łączą ich zalety. Timsort powstał w Pythonie (Tim Peters, 2002).

Ćwiczenia na zajęciach (1/3)

1. Wykonaj **dwa przebiegi** sortowania bąbelkowego dla $[4, 3, 2, 1, 5]$.
2. Posortuj kubełkowo (szerokość kubełka 10) listę $[31, 7, 22, 45, 14, 38, 9, 26]$: podaj zawartość kubełków i wynik.
3. Kiedy sortowanie kubełkowe działa wolno? Podaj przykład danych.
4. Wykonaj sortowanie przez wybieranie dla $[64, 25, 12, 22, 11]$ – podaj stan po każdym kroku.

Ćwiczenia na zajęciach (2/3)

5. Wykonaj sortowanie przez wstawianie dla $[7, 3, 9, 1]$ – podaj stan po każdym kroku.
6. Dla listy $[4, 1, 6, 3, 5, 2]$ wykonaj pierwszy podział quicksort (pivot = element środkowy `a[len(a)//2]`): podaj `mniejsze`, `rowne`, `wieksze`.
7. Dla listy $[4, 1, 6, 3, 5, 2]$ wykonaj podział w miejscu (schemat Lomuto, pivot = ostatni element): podaj listę po podziale i indeks pivota.
8. Scal listy $[1, 4, 7]$ i $[2, 3, 9]$ i policz porównania.

Ćwiczenia na zajęciach (3/3)

9. a) Ile porównań wykona sortowanie przez wybieranie dla $n = 10$?
   b) Ile zamian wykona sortowanie bąbelkowe dla listy malejącej o $n = 6$ elementach?
10. Który z poznanych algorytmów jest najlepszy dla listy prawie posortowanej? Który ma gwarantowane $O(n \log n)$? Które (w standardowych wersjach) są niestabilne?
11. Jakiego algorytmu używa `sorted()` w Pythonie i czy jest stabilny?

Praca domowa (1/4)

1. Zaimplementuj w Pythonie **wszystkie poznane sortowania**: bąbelkowe (z flagą), kubełkowe, przez wybieranie, przez wstawianie, szybkie (wersja z listami), szybkie w miejscu (z zamianami) oraz przez scalanie. Sprawdź je na liście $[6, 3, 8, 2, 9, 1]$ (kubełkowe dodatkowo na $[52, 8, 37, 14, 29, 3, 41, 18]$).
2. Wykonaj sortowanie bąbelkowe dla $[6, 3, 8, 2, 9, 1]$ – podaj stan listy po każdym przebiegu i łączną liczbę zamian.
3. Posortuj kubełkowo (szerokość kubełka 10) listę $[52, 8, 37, 14, 29, 3, 41, 18]$. Podaj zawartość kubełków (przed sortowaniem w kubełkach) i wynik.

Praca domowa (2/4)

4. Podaj dane 6-elementowe, dla których sortowanie kubełkowe (szerokość 10, sortowanie bąbelkowe w kubełkach) działa najwolniej. Jaka jest wtedy złożoność?
5. Wykonaj sortowanie przez wybieranie dla $[6, 3, 8, 2, 9, 1]$ – podaj stan po każdym kroku.
6. Scal listy $[2, 5, 8, 11]$ i $[1, 3, 9]$. Podaj wynik i liczbę porównań.
7. Wykonaj pierwszy podział quicksort (pivot = `a[len(a)//2]`) dla $[7, 2, 9, 4, 6, 1, 8]$.

Praca domowa (3/4)

8. Ile porównań wykona sortowanie przez wybieranie dla $n = 20$? Jaka lista długości 5 jest najgorsza dla sortowania bąbelkowego i ile wykona ona zamian?
9. **(stabilność)** Posortuj pary $[(2, a), (1, x), (2, b), (1, y)]$ rosnąco po pierwszej liczbie: a) stabilnie, b) sortowaniem przez wybieranie (wersja z zamianą). Czy wyniki się różnią?
10. Napisz funkcję `czy_posortowana(a)` zwracającą `True`, jeśli lista jest posortowana rosnąco. Podaj jej złożoność.

Praca domowa (4/4)

11. **(dla chętnych)** Napisz funkcję `inwersje(a)` zliczającą inwersje metodą „każda para”. Ile inwersji ma $[5, 2, 4, 6, 1, 3]$? Jaka jest jej złożoność?
12. Wykonaj podział w miejscu (schemat Lomuto, pivot = ostatni element) dla $[7, 2, 9, 4, 6, 1, 8]$. Podaj listę po podziale i indeks pivota.
13. Dlaczego biblioteki standardowe łączą kilka algorytmów sortowania (np. scalanie lub quicksort ze wstawianiem)?

Dziękuję za uwagę!